

Linguaggi Formali e Compilatori

Argomento 06: Semantica dei Linguaggi

Prof. Giuseppe Psaila

Laurea Magistrale in Ingegneria Informatica
Università di Bergamo

- Le grammatiche BNF hanno la capacità di specificare strutture sintattiche complesse, che i parser deterministici riescono a riconoscere in molti casi
- Ma trattano i terminali solo in base alla loro categoria lessicale, non in base al loro valore
- In altre parole, non viene considerato il **Significato** delle frasi del linguaggio (la **semantica del linguaggio**)

- Che cosa è il **Significato** di una frase?
- Dipende dal linguaggio
- Nelle espressioni matematiche, il significato è il **Valore** dell'espressione
- Nei linguaggi di programmazione procedurale, il significato è la versione in linguaggio macchina

- **Correttezza Semantica:** va oltre la correttezza **Sintattica**, perché per essere verificata occorre usare i valori dei terminali
- Nell'XML, la correttezza semantica è necessaria per verificare se i marcatori di apertura e chiusura corrispondenti hanno lo stesso nome
- Nei linguaggi di programmazione procedurale, la correttezza semantica è necessaria per gestire le definizioni delle variabili prima del loro uso e i tipi delle espressioni

- Per gestire gli aspetti semantici (la **Semantica del Linguaggio**)
- occorre introdurre un nuovo formalismo,
- che superi i limiti delle grammatiche BNF
- Questo formalismo prende il nome di
GRAMMATICHE AD ATTRIBUTI

Approccio

- Non si buttano via le grammatiche BNF, perchè fanno egregiamente il loro lavoro
- Le grammatiche ad attributi vengono **Aggunte** alle grammatiche BNF
- Infatti, le grammatiche BNF e gli alberi sintattici forniscono lo **Scheletro** sul quale fare la **Valutazione Semantica**

Attributo

- Un **Attributo** a associato all'occorrenza di un **Non-Terminale** N
denota una proprietà dell'occorrenza di quel non-terminale
- Con $Attr(N)$ indichiamo l'**Insieme degli Attributi del Non-Terminale** N .
- Nell'albero sintattico, tutte le occorrenze N_i dello stesso Non-Terminale N hanno lo stesso insieme di attributi, cioè $Attr(N_i) = Attr(N)$.

Regola Semantica

- Si consideri una **Regola di Produzione**
 $p : X_0 \rightarrow X_1 X_2 \dots X_h$
- Alla regola di produzione p si associa un insieme di **REGOLE SEMANTICHE** $R(p)$
- Una **REGOLA SEMANTICA** ha la forma
 $a \text{ of } X_i := f(a_1 \text{ of } X_{p_1}, \dots, a_j \text{ of } X_{p_j}, \dots a_n \text{ of } X_{p_n})$
dove $i, p_j \in \{0, 1, 2, \dots, h\}$
 $a \in Attr(X_i)$ e $a_j \in Attr(X_{p_j})$
- f è detta **Funzione Semantica**.

Regola Semantica

Notazione alternativa

- Data la regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$
- La regola semantica

$a \text{ of } X_i := f(a_1 \text{ of } X_{p1}, \dots, a_j \text{ of } X_{pj}, \dots, a_n \text{ of } X_{pn})$

può essere scritta anche come

$$X[i].a := f(X[p1].a_1, \dots, X[pj].a_j, \dots, X[pn].a_{pn})$$

Attributi Ereditati

- Data la regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$
- e data la regola semantica
 $a \text{ of } X_i := f(a_1 \text{ of } X_{p1}, \dots, a_j \text{ of } X_{pj}, \dots a_n \text{ of } X_{pn})$
- l'attributo $a \text{ of } X_i$ è detto **EREDITATO** se $i \geq 1$
(cioè l'attributo a appartiene ad un **figlio** della regola di produzione).

Attributi Sintetizzati

- Data la regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$
- e data la regola semantica
 $a \text{ of } X_i := f(a_1 \text{ of } X_{p1}, \dots, a_j \text{ of } X_{pj}, \dots a_n \text{ of } X_{pn})$
- l'attributo $a \text{ of } X_i$ è detto **SINTETIZZATO** se $i = 0$
(cioè l'attributo a appartiene al **padre** della regola di produzione).

Attributi Ereditati e Sintetizzati

- Dato un Non-Terminale N , indichiamo con $Ere(N) = \{a_1, a_2, \dots\}$ l'insieme dei suoi attributi ereditati, cioè l'insieme degli attributi $a \in Attr(N)$ per cui esista almeno una regola semantica $a \text{ of } N_i := \dots$, con $i \geq 1$.
- Dato un Non-Terminale N , indichiamo con $Sin(N) = \{a_1, a_2, \dots\}$ l'insieme dei suoi attributi sintetizzati, cioè l'insieme degli attributi $a \in Attr(N)$ per cui esista almeno una regola semantica $a \text{ of } N_i := \dots$, con $i = 0$.

Grammatiche ad Attributi Ben Formate

- Per ogni Non-Terminale $N \in V$,
 $Ere(N) \cup Sin(N) = Attr(N)$
 $Ere(N) \cap Sin(N) = \emptyset$
- Per ogni regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$,
per ogni Non-Terminale figlio $X_i \in V$ (con $i \geq 1$),
per ogni attributo $a \in Ere(X_i)$ deve esserci **una e una sola** regola semantica che lo definisce;
- Per ogni regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$,
per ogni attributo $a \in Sin(X_0)$ deve esserci **una e una sola** regola semantica che lo definisce.

Attributi dei Terminali

- Dal punto di vista sintattico, non ha importanza conoscere l'effettiva stringa associata all'occorrenza di un simbolo terminale
- Ma dal punto di vista semantico, spesso è necessario (pensate ad XML o ai nomi delle variabili).
- Ogni simbolo terminale $t \in \Sigma$ ha un **Attributo Predefinito** denominato `Value`, che indica la stringa di testo associata all'occorrenza del terminale.
- Possiamo dire che $Sin(t) = \{Value\}$ e che $Ere(t) = \emptyset$.

Esempio

- 1) $S \rightarrow L$
- 2) $L \rightarrow A L$
- 3) $L \rightarrow B L$
- 4) $L \rightarrow \epsilon$
- 5) $A \rightarrow a$
- 6) $B \rightarrow b$

Vogliamo scrivere una grammatica ad attributi che conta il numero di a e di b

Impostiamo gli Attributi

- $Attr(L) = \{tota, totb\}$
- $Attr(S) = \{tota, totb\}$
- $Attr(A) = \emptyset$
- $Attr(B) = \emptyset$

- 1) $S \rightarrow L$

$S[0].tota := L[1].tota$

$S[0].totb := L[1].totb$

- 2) $L \rightarrow A L$

$L[0].tota := L[2].tota + 1$

$L[0].totb := L[2].totb$

- 3) $L \rightarrow B L$

$L[0].tota := L[2].tota$

$L[0].totb := L[2].totb + 1$

- 4) $L \rightarrow \epsilon$
 $L[0].tota := 0$
 $L[0].totb := 0$
- 5) $A \rightarrow a$
- 6) $B \rightarrow b$

- Abbiamo usato la funzione semantica “+”, con la forma infissa
- Se vogliamo vederla in forma funzionale, possiamo introdurre la funzione Increase
- 2) $L \rightarrow A L$
 $L[0].tota := \text{Increase}(L[2].tota)$
 $L[0].totb := L[2].totb$
- 3) $L \rightarrow B L$
 $L[0].tota := L[2].tota$
 $L[0].totb := \text{Increase}(L[2].totb)$

- Alcune regole non usano alcuna funzione semantica
- 3) $L \rightarrow B L$
 $L[0].tota := L[2].tota$
- Sono dette **Regole di Propagazione**, perché propagano il valore di un attributo ad un altro attributo, senza alterarlo.

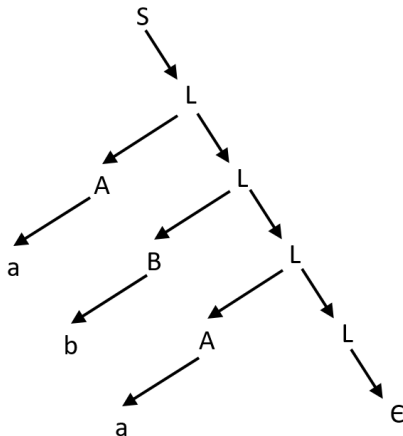
Dipendenza Funzionale

- Dato un albero sintattico e l'occorrenza di una regola di produzione che genera le occorrenze dei Non-Terminali e Terminali $p : X_0 \rightarrow X_1 X_2 \dots X_h$, si dice che a of X_i **DIPENDE FUNZIONALMENTE** da a_j of X_{pj} , indicato come a of $X_i \leftarrow a_j$ of X_{pj} (o anche come a_j of $X_{pj} \rightarrow a$ of X_i), se esiste una regola semantica associata a p tale che a of $X_i := f(\dots, a_j$ of $X_{pj}, \dots)$.
- a of $X_i \leftarrow a_j$ of X_{pj} viene detta **DIPENDENZA FUNZIONALE**.

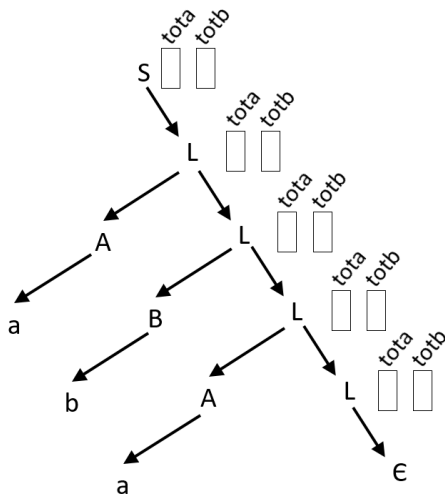
Albero Sintattico Decorato

- Data una stringa s e il corrispondente albero sintattico
- L'albero sintattico esteso con le **Occorrenze degli Attributi** e con le **Dipendenze Funzionali** indotte dalle regole semantiche viene detto **ALBERO DECORATO**

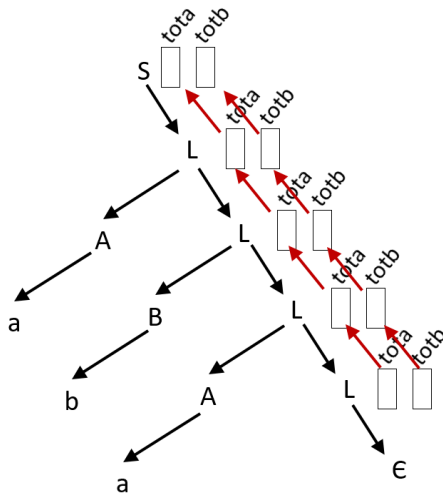
Albero Sintattico Decorato



Albero Sintattico Decorato



Albero Sintattico Decorato



Valutazione Semantica

- Gli attributi presenti nell'albero sintattico decorato devono essere **Valutati**,
cioè occorre valutare il loro valore, applicando le regole semantiche.
- Trascrivendo le regole semantiche, applicandole alle specifiche occorrenze degli attributi, otteniamo un **Sistema di EQUAZIONI SEMANTICHE**.
- Valutare gli attributi vuole dire
Risolvere il Sistema di Equazioni Semantiche.

Valutazione Semantica

- In termini pratici, le **Dipendenze Funzionali** forniscono la chiave per risolvere il sistema di equazioni semantiche
- Infatti, se calcoliamo l'**Ordinamento Topologico** delle occorrenze degli attributi sull'albero sintattico decorato in base alle dipendenze funzionali, otteniamo l'**Ordine** con cui le occorrenze degli attributi devono essere valutate

Valutazione Semantica

- Tuttavia, se il sistema di equazioni semantiche dà luogo a **Dipendenze Circolari**
- il sistema di equazioni semantiche
NON HA ALCUNA SOLUZIONE

Valutazione Semantica

- Condizione necessaria e sufficiente perché il sistema di equazioni semantiche abbia una e una sola soluzione è che la grammatica ad attributi sia ben formata e che le dipendenze funzionali non presentino circolarità

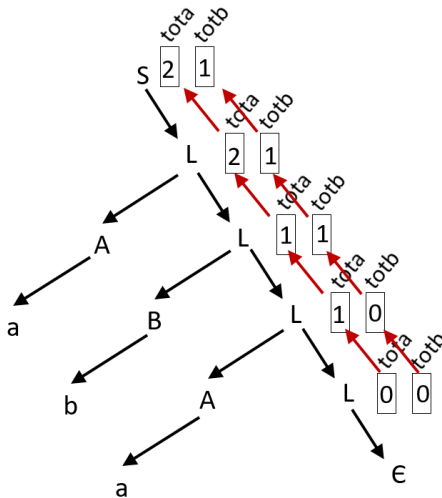
Valutazione Semantica

- Se si riesce a dimostrare che, per qualsiasi albero sintattico, il sistema di equazioni semantiche non presenta circolarità,
- allora la valutazione semantica è **SEMPRE POSSIBILE**.
- In tal caso, la Grammatica ad Attributi viene detta **Aciclica**.

Valutazione Semantica Base

- Dato l'albero sintattico decorato D
- si calcola l'ordinamento topologico $O = \langle a_1, a_2, \dots \rangle$ (sequenza), in base alle dipendenze funzionali.
- Ogni attributo $a_i \in O$ viene valutato applicando la corrispondente regola semantica, seguendo l'ordine con cui compare in O .

Albero Sintattico Decorato



Osservazioni

- Gli attributi $tota$ e $totb$ sono sintetizzati.
Non ci sono attributi ereditati.
Questa grammatica viene detta
Puramente Sintetizzata.

Variante

Adottiamo un approccio **Discendente**

- $Attr(L) = \{eta, etb, sta, stb\}$
- $Attr(S) = \{tota, totb\}$
- $Attr(A) = \emptyset$
- $Attr(B) = \emptyset$

- 1) $S \rightarrow L$

$S[0].tota := L[1].sta$

$S[0].totb := L[1].stb$

$L[1].eta := 0$

$L[1].etb := 0$

- 2) $L \rightarrow A L$

$L[0].sta := L[2].sta$

$L[0].stb := L[2].stb$

$L[2].eta := L[0].eta + 1$

$L[2].etb := L[0].etb$

- 3) $L \rightarrow B L$

$L[0].sta := L[2].sta$

$L[0].stb := L[2].stb$

$L[2].eta := L[0].eta$

$L[2].etb := L[0].etb + 1$

- 4) $L \rightarrow \epsilon$

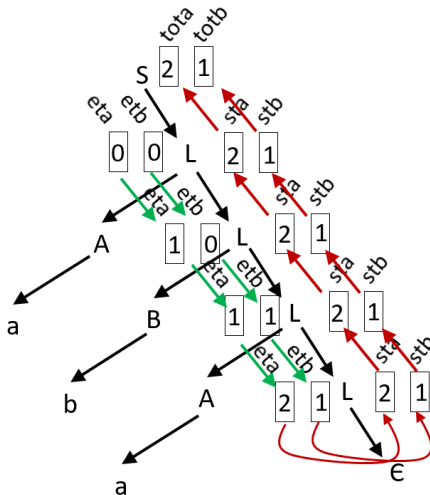
$L[0].sta := L[0].eta$

$L[0].stb := L[0].etb$

- 5) $A \rightarrow a$

- 6) $B \rightarrow b$

Albero Sintattico Decorato



Osservazioni

- Per mantenere una certa chiarezza nel disegno, conviene adottare una semplice convenzione:
Gli attributi ereditati vanno a sinistra del non-terminale
Gli attributi sintetizzati vanno a destra del non-terminale
- Se possibile, conviene usare colori diversi per le frecce:
le dipendenze che valutano attributi ereditati di un colore (io ho usato il verde),
le dipendenze che valutano gli attributi sintetizzati di un altro colore (io ho usato il rosso).

Gestione delle Variabili in un Linguaggio di Programmazione

- 1) $PR \rightarrow LI$
- 2) $LI \rightarrow I \ LI$
- 3) $LI \rightarrow \epsilon$
- 4) $I \rightarrow VDEF$
- 5) $I \rightarrow VUSE$
- 6) $VDEF \rightarrow Def \ Name$
- 7) $VUSE \rightarrow Name$

Gestione delle Variabili in un Linguaggio di Programmazione

- Il programma è una lista di istruzioni
- Un'istruzione è
 - o una definizione di variabile
 - o un uso di variabile
- Una definizione di variabile inizia con la parola chiave "DEF" (terminale Def) seguita dal nome (terminale Name)
- L'uso di una variabile è dato dal solo nome della variabile (terminale Name)

Controlli Semantici

- Verificare che tutte le variabili usate siano state precedentemente definite
- Approccio: Dobbiamo costruire la **Tabella delle Variabili**
- che useremo per controllare che la variabile usata sia definita
- La condizione di errore è il risultato della valutazione (attributo dell'assioma).
- Gli errori devono essere segnalati nell'ordine naturale.

Attributi

- $Ere(PR) = \emptyset$
 $Sin(PR) = \{err\}$
- $Ere(LI) = \{ev\}$
 $Sin(LI) = \{err\}$
- $Ere(I) = \{ev\}$
 $Sin(I) = \{err, sv\}$
- $Ere(VDEF) = \{ev\}$
 $Sin(VDEF) = \{err, sv\}$
- $Ere(VUSE) = \{ev\}$
 $Sin(VUSE) = \{err\}$

- 1) $PR \rightarrow LI$
 $LI[1].ev := \emptyset$
 $PR[0].err := LI[1].err$
- 2) $LI \rightarrow I LI$
 $I[1].ev := LI[0].ev$
 $LI[2].ev := I[1].sv$
 $LI[0].err := I[1].err \text{ OR } LI[2].err$
- 3) $LI \rightarrow \epsilon$
 $LI[0].err := \text{False}$

- 4) $I \rightarrow VDEF$

$VDEF[1].ev := I[0].ev$

$I[0].sv := VDEF[1].sv$

$I[0].err := VDEF[1].err$

- 5) $I \rightarrow VUSE$

$VUSE[1].ev := I[0].ev$

$I[0].sv := I[0].ev$

$I[0].err := VUSE[1].err$

- 6) $VDEF \rightarrow Def \ Name$

$VDEF[0].err :=$

$CheckDef(VDEF[0].ev, Name[2].Value)$

$VDEF[0].sv :=$

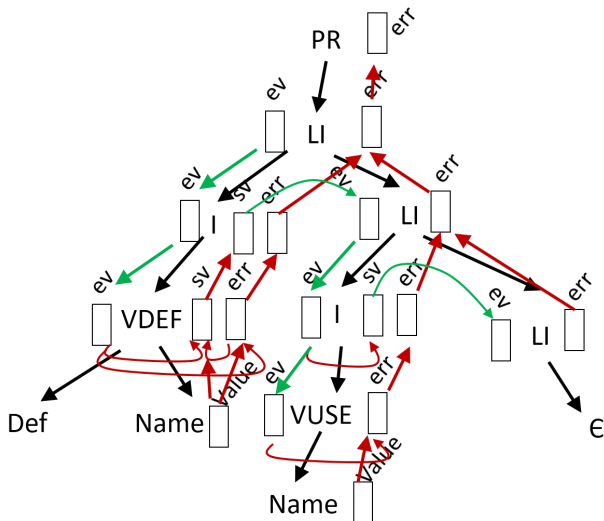
$AddDef(VDEF[0].ev, Name[2].Value,$
 $VDEF[0].err)$

- 7) $VUSE \rightarrow Name$

$VUSE[0].err :=$

$CheckUse(VUSE[0].ev, Name[1].Value)$

Albero Sintattico Decorato



Correttezza dei Documenti XML

- 1) $S \rightarrow E$
- 2) $E \rightarrow o A e$
- 3) $E \rightarrow o A n C c$
- 4) $A \rightarrow a u s A$
- 5) $A \rightarrow \epsilon$
- 6) $C \rightarrow t C$
- 7) $C \rightarrow E C$
- 8) $C \rightarrow \epsilon$

Correttezza dei Documenti XML

- Vogliamo controllare che il nome del marcatore di chiusura (terminale c) sia lo stesso del corrispondente marcatore di apertura (terminale o).
- Vogliamo controllare che un elemento non abbia nomi di attributi duplicati.

Attributi

- $Ere(S) = \emptyset$
 $Sin(S) = \{err\}$
- $Ere(E) = \emptyset$
 $Sin(E) = \{err, bad\}$
- $Ere(C) = \emptyset$
 $Sin(C) = \{err\}$
- $Ere(A) = \{attrs\}$
 $Sin(A) = \{err, dup\}$

- 1) $S \rightarrow E$
 $S[0].err := E[1].err$
- 2) $E \rightarrow o A e$
 $E[0].err := A[2].err$
 $A[2].attrs := \emptyset$
 $E[0].bad := False$
- 3) $E \rightarrow o A n C c$
 $A[2].attrs := \emptyset$
 $E[0].bad :=$
 $CheckNames(o[1].Value, c[5].Value)$
 $E[0].err :=$
 $A[2].err \text{ OR } C[4].err \text{ OR } E[0].bad$

- 4) $A \rightarrow a \ u \ s \ A$
 $A[0].dup :=$
 $CheckAttr(A[0].attrs, a[1].value)$
 $A[4].attrs := AddAttr(A[0].attrs,$
 $a[1].Value, A[0].dup)$
 $A[0].err := A[0].dup \ OR \ A[4].err$
- 5) $A \rightarrow \epsilon$
 $A[0].err := False$
 $A[0].dup := False$

- 6) $C \rightarrow t \ C$
 $C[0].err := C[2].err$
- 7) $C \rightarrow E \ C$
 $C[0].err := E[1].err \text{ OR } C[2].err$
- 8) $C \rightarrow \epsilon$
 $C[0].err := \text{False}$

Valutazione Efficiente

- Il metodo di valutazione generale delle gramamtiche ad attributi è efficace ma non efficiente
- Sono state identificate delle tecniche di valutazione efficiente, adatte a specifiche **Famiglie** di Grammatiche ad Attributi.

Valutazione One-Sweep

- La valutazione One-Sweep (a una spazzata dell'albero) è molto efficiente,
- perché viene fatta facendo una sola visita dell'albero sintattico
- Il caso particolare detto **TIPO L** può essere valutato durante il parsing discendente.

Condizioni One-Sweep

- **Condizione 1**

Per ogni regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$, non devono esserci regole semantiche del tipo

$a \text{ of } X_i := f(\dots, a_j \text{ of } X_{pj}, \dots)$.

con $a \text{ of } X_i \in \text{Ere}(X_i)$, con $i \geq 1$

e con $a_j \text{ of } X_{pj} \in \text{Sin}(X_{pj})$, con $pj = 0$

Condizioni One-Sweep

- Per ogni regola di produzione $p : X_0 \rightarrow X_1 X_2 \dots X_h$, si calcola il **BROTHER GRAPH** $BG = (N, E)$

Nodi $N = \{X_1, \dots, X_h\}$

Archi $E = \{X_{pj} \rightarrow X_i, \dots\}$ se esiste una regola semantica

$a \text{ of } X_i := f(\dots, a_j \text{ of } X_{pj}, \dots)$,

con $i, pj > 0$.

- Non si considerano le dipendenze tali che
 - $i = pj$ e $a_j \in Ere(X_{pj})$ (da ereditato ad ereditato dello stesso figlio)
 - mentre si considerano da sintetizzato ad ereditato dello stesso figlio, che crea un anello sul nodo

Condizioni One-Sweep

- **Condizione 2**

Si verifica se il Brother Graph è **Aciclico**, calcolando l'ordinamento topologico $O(p)$:
se non lo è, si genera errore

- La grammatica ad attributi è **One-Sweep** se le condizioni 1 e 2 valgono per tutte le regole di produzione

Valutazione One-Sweep

- Per ogni regola di produzione p , prendiamo l'ordinamento topologico $O(p) = \langle X_{p1}, X_{p2}, \dots \rangle$
- Una procedura che visita il non-terminale X_0 in cui è radicata la regola di produzione p
Per ogni figlio $X_{pi} \in O(p)$ (nell'ordine)
 - valuta tutti gli attributi $a \in Ere(X_{pi})$ (rispettando eventuali **Dipendenze Interne**)
 - Visita X_{pi}
- Valuta tutti gli attributi sintetizzati $a \in Sin(X_0)$ (rispettando eventuali **Dipendenze Interne**)
Si termina la visita

Valutazione One-Sweep

- Cosa si intende con **rispettando eventuali dipendenze interne**?
- Prendiamo un non-terminale X_i (con $i \geq 0$) se esiste una regola semantica $a \text{ of } X_i := f(\dots, b \text{ of } X_i, \dots)$.
- Questa è una **Dipendenza Interna**: occorre valutare prima b e poi a

Esempio: XML

- La regola 4 non rispetta la condizione 1

4) $A \rightarrow a \text{ u } s \ A$

$A[0].\text{dup} :=$

$\text{CheckAttr}(A[0].\text{attrs}, a[1].\text{value})$

$A[4].\text{attrs} := \text{AddAttr}(A[0].\text{attrs},$
 $a[1].\text{Value}, A[0].\text{dup})$

$A[0].\text{err} := A[0].\text{dup} \text{ OR } A[4].\text{err}$

Brother Graphs

1) E

2) A

3) A C

4) A

5)

6) C

7) E C

8)

Grammatica Modificata

- Spostiamo l'attributo `dup` del Non-Terminale `A` da sintetizzato ad ereditato.
- $Ere(S) = \emptyset$
 $Sin(S) = \{err\}$
- $Ere(E) = \emptyset$
 $Sin(E) = \{err, bad\}$
- $Ere(C) = \emptyset$
 $Sin(C) = \{err\}$
- $Ere(A) = \{attrs, dup\}$
 $Sin(A) = \{err\}$

- 2) $E \rightarrow o A e$

$E[0].err := A[2].err$

$A[2].attrs := \emptyset$

$E[0].bad := False$

$A[2].dup := False$

- 3) $E \rightarrow o A n C c$

$A[2].attrs := \emptyset$

$E[0].bad :=$

$CheckNames(o[1].Value, c[5].Value)$

$E[0].err :=$

$A[2].err \text{ OR } C[4].err \text{ OR } E[0].bad$

$A[2].dup := False$

- 4) $A \rightarrow a \ u \ s \ A$

$A[4].dup :=$

$\text{CheckAttr}(A[0].attrs, a[1].value)$

$A[4].attrs := \text{AddAttr}(A[0].attrs,$
 $a[1].Value, A[4].dup)$

$A[0].err := A[4].dup \text{ OR } A[4].err$

- 5) $A \rightarrow \epsilon$

$A[0].err := \text{False}$

- Così la condizione 1 è soddisfatta per tutte le regole di produzione
- I Brother Graphs rimangono vuoti, quindi un qualsiasi ordinamento topologico va bene.

Grammatiche di Tipo L

- Una grammatica ad attributi è detta di **Tipo L** se è **One-Sweep**
e se ammette, per tutti i Brother Graphs,
l'ordinamento topologico $O(p) = \langle X_1, X_2, \dots, X_h \rangle$
- Le grammatiche di Tipo L possono essere valutate durante il parsing a discesa ricorsiva, senza materializzare l'albero.
- La grammatica ad attributi modificata per l'XML è di Tipo L.

Esercizio

- La grammatica ad attributi per la definizione e uso delle variabili è **One-Sweep**?
- È di **Tipo L**?

Brother Graphs

- 1) LI
- 2) 
- 4) VDEF
- 5) VUSE

Anche la condizione 1 è rispettata, quindi la grammatica è One-Seep.

In più, è di Tipo L.

Che fare se ...

- ... la grammatica ad attributi NON È ONE-SWEEP?
- Se l'albero sintattico è materializzato, si può pensare di fare **passate multiple** sull'albero
- creando delle partizioni degli attributi, in modo da fare una passata per ogni partizione.
- Queste sono le grammatiche ad attributi **Multi-Sweep** o **Multi-Pass**.

Grammatiche Multi-Sweep

- Costruire il **Grafo Semplice** (Simple Graph)
 $S = (N, E)$ delle dipendenze tra gli attributi,
indipendentemente dalla regola di produzione e dal
Non-Terminale
Nodi: $N = \{a_1, a_2, \dots\}$ (attributi)
Archi: $E = \{b \rightarrow a, \dots\}$ se esiste una regola
semantica in una regola di produzione p tale che
 $a \text{ of } X_i := f(\dots, b \text{ of } X_{ipj}, \dots)$.

Grammatiche Multi-Sweep

- Identificare le **Componenti Connesse** $C_1, C_2, \dots, C_n$, dove

$$C_i = \{a_{i,1}, a_{i,2}, \dots\} \subseteq N$$

e per ogni coppia C_i, C_j , deve essere $C_i \cap C_j = \emptyset$

Grammatiche Multi-Sweep

- Creare il **Grafo Collassato** (Collapsed Graph)

$$CG = (\overline{N}, \overline{E})$$

$$\overline{N} = \{C_1, C_2, \dots, C_n\}$$

$$\overline{E} = \{C_i \rightarrow C_j, \dots\}$$

se $b \rightarrow a \in E$ tale che $b \in C_i$ e $a \in C_j$.

- Calcolare l'ordinamento topologico

$$OC = \langle C_1, C_2, \dots \rangle \text{ del grafo collassato.}$$

- Ogni componente connessa C_i deve rispettare le condizioni One-Sweep.

La valutazione consiste di n visite dell'albero di tipo One-Sweep

Definizione Ritardata delle Variabili (late binding)

- Se il linguaggio ammette la possibilità di usare delle variabili (o, in generale, dei simboli) **Prima** che questi vengono definiti
- Il controllo che ogni variabile usata sia anche definita diventa più complesso
- Possibile approccio:
raccogliere prima tutte le definizioni di variabili,
ovunque esse siano (verificando la doppia definizione)
Poi verificare l'uso delle variabili.

Attributi

- $Ere(PR) = \emptyset$
 $Sin(PR) = \{err\}$
- $Ere(LI) = \{ev, fv\}$
 $Sin(LI) = \{derr, uerr, sv\}$
- $Ere(I) = \{ev, fv\}$
 $Sin(I) = \{derr, uerr, sv\}$
- $Ere(VDEF) = \{ev\}$
 $Sin(VDEF) = \{derr, sv\}$
- $Ere(VUSE) = \{fv\}$
 $Sin(VUSE) = \{uerr\}$

Nuovi Attributi

- Aggiungiamo l'attributo fv (full variable list)
- distinguiamo due tipi di errori:
 - derr, errore di definizione
 - uerr, errore d'uso

- 1) $PR \rightarrow LI$

$LI[1].ev := \emptyset$

$PR[0].err := LI[1].derr \text{ OR } LI[1].uerr$

$LI[1].fv = LI[1].sv$

- 2) $LI \rightarrow I \ LI$

$I[1].ev := LI[0].ev$

$LI[2].ev := I[1].sv$

$LI[0].derr := I[1].derr \text{ OR } LI[2].derr$

$LI[0].uerr := I[1].uerr \text{ OR } LI[2].uerr$

$LI[0].sv := LI[2].sv$

$I[1].fv := LI[0].fv$

$LI[2].fv := LI[0].fv$

- 3) $LI \rightarrow \epsilon$

`LI[0].derr := False`

`LI[0].uerr := False`

`LI[0].sv := LI[0].ev`

- 4) $I \rightarrow VDEF$

$VDEF[1].ev := I[0].ev$

$I[0].sv := VDEF[1].sv$

$I[0].derr := VDEF[1].derr$

$I[0].uerr := False$

- 5) $I \rightarrow VUSE$

$VUSE[1].fv := I[0].fv$

$I[0].sv := I[0].ev$

$I[0].uerr := VUSE[1].uerr$

$I[0].derr := False$

- 6) $VDEF \rightarrow Def \ Name$

$VDEF[0].derr :=$

$CheckDef(VDEF[0].ev, Name[2].Value)$

$VDEF[0].sv :=$

$AddDef(VDEF[0].ev, Name[2].Value,$
 $VDEF[0].derr)$

- 7) $VUSE \rightarrow Name$

$VUSE[0].uerr :=$

$CheckUse(VUSE[0].fv, Name[1].Value)$

Brother Graphs Vi è un ciclo: non è One-Sweep



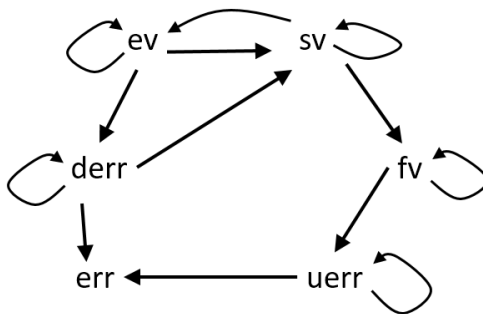
4)

VDEF

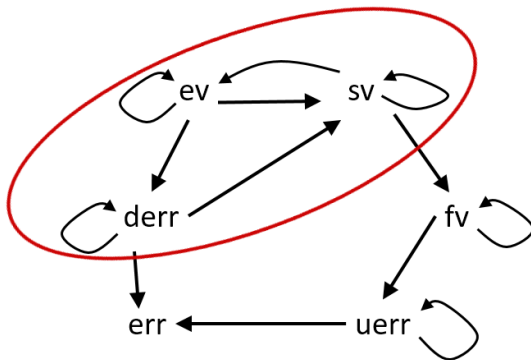
5)

VUSE

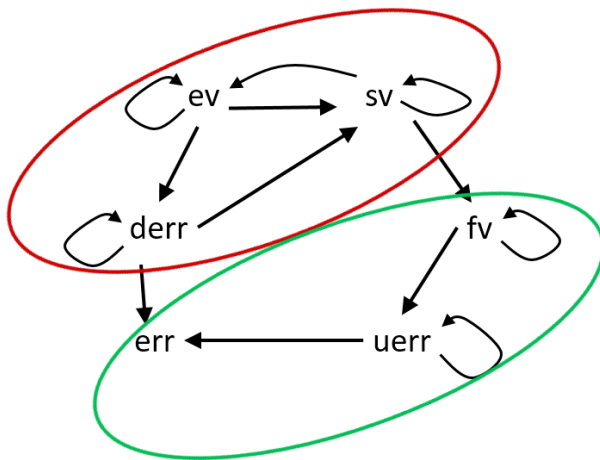
Simple Graph



Componenti Connesse



Componenti Connesse



Collapsed Graph

$C1 \longrightarrow C2$

- $C1 = \{ev, sv, derr\}$, prima passata (colore ROSSO)
- $C2 = \{fv, uerr, err\}$, seconda passata (colore VERDE)

- 1) $PR \rightarrow LI$

$LI[1].ev := \emptyset$

$PR[0].err := LI[1].derr \text{ OR } LI[1].uerr$

$LI[1].fv = LI[1].sv$

- 2) $LI \rightarrow I \ LI$

$I[1].ev := LI[0].ev$

$LI[2].ev := I[1].sv$

$LI[0].derr := I[1].derr \text{ OR } LI[2].derr$

$LI[0].uerr := I[1].uerr \text{ OR } LI[2].uerr$

$LI[0].sv := LI[2].sv$

$I[1].fv := LI[0].fv$

$LI[2].fv := LI[0].fv$

- 3) $LI \rightarrow \epsilon$

$LI[0].derr := \text{False}$

$LI[0].uerr := \text{False}$

$LI[0].sv := LI[0].ev$

- 4) $I \rightarrow VDEF$

$VDEF[1].ev := I[0].ev$

$I[0].sv := VDEF[1].sv$

$I[0].derr := VDEF[1].derr$

$I[0].uerr := \text{False}$

- 5) $I \rightarrow VUSE$

$VUSE[1].fv := I[0].fv$

$I[0].sv := I[0].ev$

$I[0].uerr := VUSE[1].uerr$

$I[0].derr := \text{False}$

- 6) $VDEF \rightarrow Def \ Name$

```
VDEF[0].derr :=  
    CheckDef(VDEF[0].ev, Name[2].Value)  
VDEF[0].sv :=  
    AddDef(VDEF[0].ev, Name[2].Value,  
          VDEF[0].derr)
```

- 7) $VUSE \rightarrow Name$

```
VUSE[0].uerr :=  
    CheckUse(VUSE[0].fv, Name[1].Value)
```


- Le tecniche di valutazione semantica lavorano con casi particolari di grammatiche **Acicliche**, cioè che non presentano circolarità nel grafo delle dipendenze per nessun albero sintattico.
- Ma è possibile stabilire se una grammatica è **Aciclica** in modo generale?
- Donald Knuth, l'inventore delle grammatiche ad attributi, ha affrontato fin da subito il problema ma al primo tentativo ha sbagliato, l'algoritmo proposto non era esatto

- Esistono quindi due algoritmi (entrambi proposti da Knuth):
 - **Aciclicità Assoluta** (il primo)
 - **Aciclicità Esatta** (quello corretto)

Passo 0

- Per ogni regola di produzione $p : A_0 \rightarrow A_1 A_2 \dots A_n$
Dato il grafo delle dipendenze funzionali Dip_p , se ne calcola la sua **Chiusura Transitiva** $(Dip_p)^+$
Si estrae $Dip_p^0(A_0)$, l'insieme degli archi
 η of $A_0 \rightarrow \sigma$ of A_0 ($\eta \in Ere(A_0)$, $\sigma \in Sin(A_0)$).
- Per ogni simbolo non-terminale $A \in V$, si calcola
 $D^0(A) = \cup_p Dip_p^0(A_0)$ per tutte le regole di produzione
 p con $A_0 = A$.

Passo Iterativo $i \geq 1$

- Per ogni regola di produzione $p : A_0 \rightarrow A_1 A_2 \dots a_n$ e per ogni non-terminale figlio A_j (con $1 \leq j \leq n$)
Si calcola $Dip'_p = Dip_p \cup_j D^{i-1}(A_j)$
e la sua chiusura transitiva $(Dip'_p)^+$
da cui si estrae $Dip^i_p(A_0)$.
- Per ogni non-terminale A , si calcola
 $D^i(A) = \cup_p Dip^i_p(A_0)$, per le regole p con $A = A_0$.
- Se succede che un $(Dip'_p)^+$ contiene cicli,
 $\Rightarrow$ La grammatica non è aciclica.
- Se per almeno un non-terminale A succede che
 $D^i(A) \neq D^{i-1}(A)$, si ripete il passo iterativo
altrimenti STOP (la grammatica è aciclica)

① $S \rightarrow A B B$

$A[1].x := f1(S[0].x)$

$B[2].x := f2(A[1].y, B[3].y)$

$B[3].x := A[1].y$

$S[0].y := f1(B[2].y)$

② $A \rightarrow a$

$A[0].y := f1(A[0].x)$

③ $B \rightarrow b$

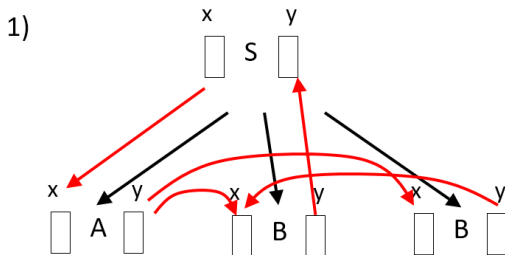
$B[0].y := 0$

④ $B \rightarrow S b$

$B[0].y := f2(B[0].x, S[1].y)$

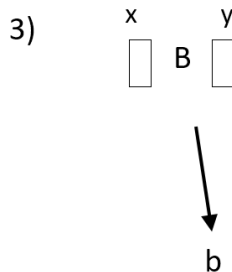
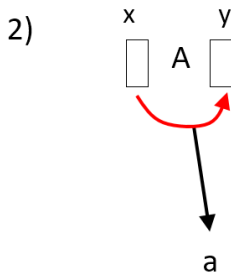
$S[1].x := f1(B[0].x)$

Passo 0



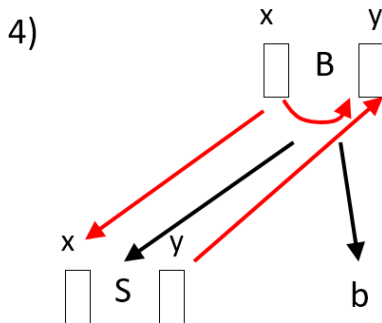
$$Dip_1^0(S) = \emptyset$$

Passo 0



$$Dip_2^0(A) = \{x \rightarrow y\}$$
$$Dip_3^0(B) = \emptyset$$

Passo 0

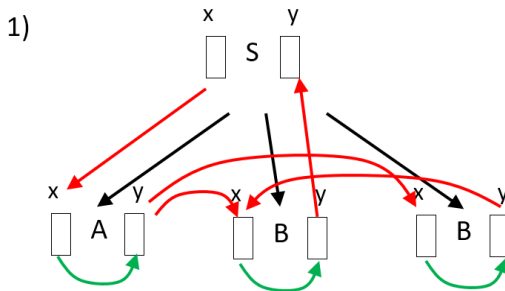


$$Dip_4^0(B) = \{x \rightarrow y\}$$

Passo 0

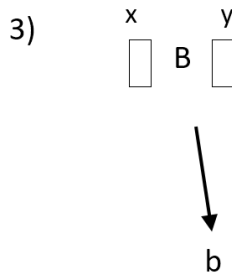
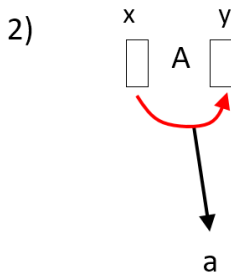
- $D^0(S) = Dip_1^0(S) = \emptyset$
- $D^0(A) = Dip_2^0(A) = \{x \rightarrow y\}$
- $D^0(B) = Dip_3^0(B) \cup Dip_4^0(B) = \emptyset \cup \{x \rightarrow y\} = \{x \rightarrow y\}$

Passo 1



$$Dip_1^1(S) = \{x \rightarrow y\}$$

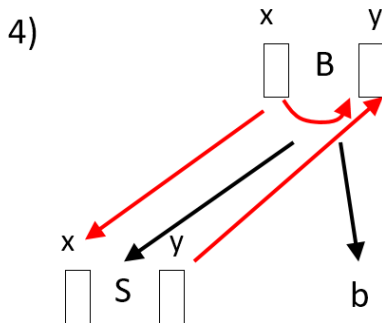
Passo 1



$$Dip_2^1(A) = \{x \rightarrow y\}$$

$$Dip_3^1(B) = \emptyset$$

Passo 1

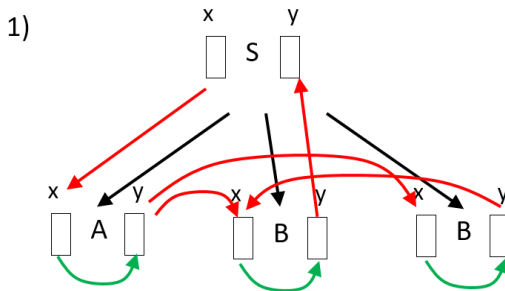


$$Dip_4^1(B) = \{x \rightarrow y\}$$

Passo 1

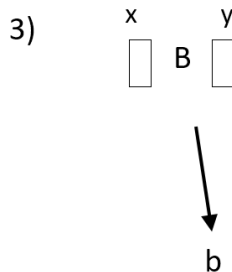
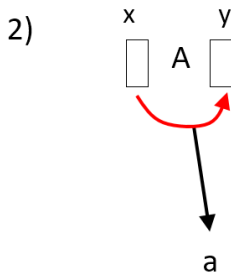
- $D^1(S) = Dip_1^1(S) = \{x \rightarrow y\}$ Cambiamento
- $D^1(A) = Dip_2^1(A) = \{x \rightarrow y\}$
- $D^1(B) = Dip_3^1(B) \cup Dip_4^1(B) = \emptyset \cup \{x \rightarrow y\} = \{x \rightarrow y\}$

Passo 2



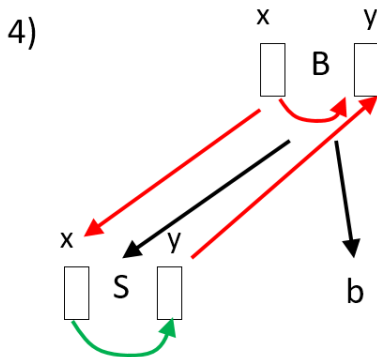
$$Dip_1^2(S) = \{x \rightarrow y\}$$

Passo 2



$$Dip_2^2(A) = \{x \rightarrow y\}$$
$$Dip_3^2(B) = \emptyset$$

Passo 2



$$Dip_4^2(B) = \{x \rightarrow y\}$$

Passo 2

- $D^2(S) = Dip_1^2(S) = \{x \rightarrow y\}$
- $D^2(A) = Dip_2^2(A) = \{x \rightarrow y\}$
- $D^2(B) = Dip_3^2(B) \cup Dip_4^2(B) = \emptyset \cup \{x \rightarrow y\} = \{x \rightarrow y\}$

Nessun ciclo e nessun cambiamento: la grammatica è aciclica

① $S \rightarrow Z$

$Z[1].a := S[0].a$

$S[0].c := Z[1].c$

② $Z \rightarrow A Z$

$A[1].a := Z[0].a$

$Z[2].a := f2(Z[0].a, A[1].c)$

$Z[0].c := Z[2].c$

③ $Z \rightarrow A$

$A[1].a := Z[0].a$

$Z[0].c := f2(Z[0].a, A[1].c)$

4 $A \rightarrow B$

$B[1].a := f2(A[0].a, B[1].c)$

$B[1].b := B[1].d$

$A[0].c := f2(B[1].c, B[1].d)$

5 $B \rightarrow 2$

$B[0].c := f1(B[1].b)$

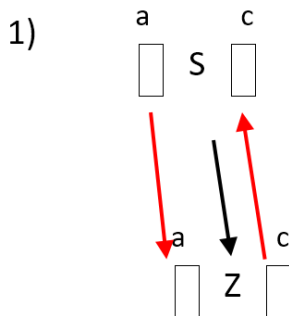
$B[0].d := 0$

6 $B \rightarrow 3$

$B[0].c := 0$

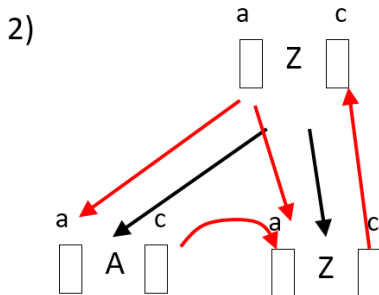
$B[0].d := f1(B[1].a)$

Passo 0



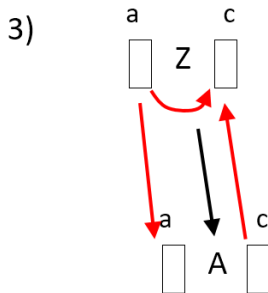
$$Dip_1^0(S) = \emptyset$$

Passo 0



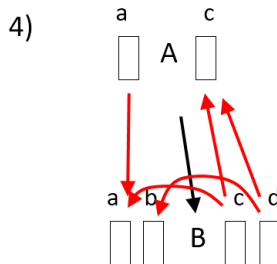
$$Dip_2^0(Z) = \emptyset$$

Passo 0



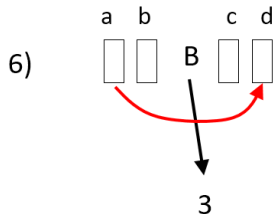
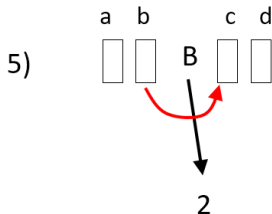
$$Dip_3^0(Z) = \{a \rightarrow c\}$$

Passo 0



$$Dip_4^0(A) = \emptyset$$

Passo 0



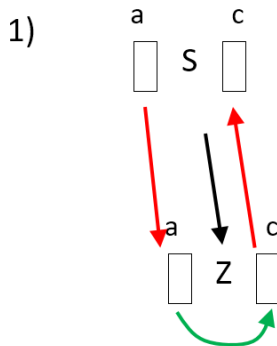
$$Dip_5^0(B) = \{b \rightarrow c\}$$

$$Dip_6^0(B) = \{a \rightarrow d\}$$

Passo 0

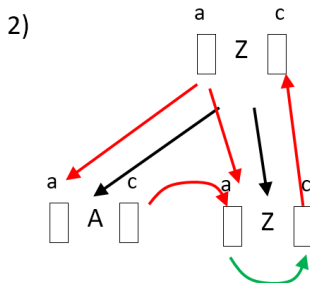
- $D^0(S) = Dip_1^0(S) = \emptyset$
- $D^0(Z) = Dip_2^0(Z) \cup Dip_3^0(Z) = \emptyset \cup \{a \rightarrow c\} = \{a \rightarrow c\}$
- $D^0(A) = Dip_4^0(A) = \emptyset$
- $D^0(B) = Dip_5^0(B) \cup Dip_6^0(B) = \{b \rightarrow c\} \cup \{a \rightarrow d\} = \{b \rightarrow c, a \rightarrow d\}$

Passo 1



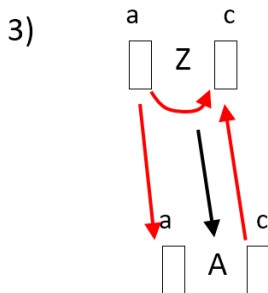
$$Dip_1^1(S) = \{a \rightarrow c\}$$

Passo 1



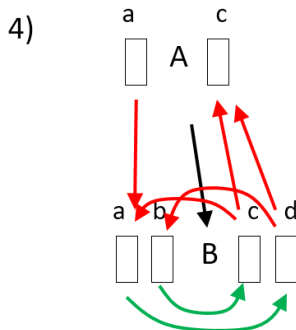
$$Dip_2^1(Z) = \{a \rightarrow c\}$$

Passo 1



$$Dip_3^1(Z) = \{a \rightarrow c\}$$

Passo 1



$$Dip_4^1(A) = \{a \rightarrow c\}$$

ATTENZIONE !!! CICLO

- La grammatica non è **Assolutamente Aciclica**
- Ma la circolarità è data dalla fusione delle dipendenze relative al non-terminale B
- Tuttavia, queste dipendenze non possono mai essere presenti insieme
- Knuth ha quindi modificato l'algoritmo, proponendo l'algoritmo esatto e chiamando il precedente Aciclicità Assoluta.

Passo 0

- Per ogni regola di produzione $p : A_0 \rightarrow A_1 A_2 \dots A_n$
Dato il grafo delle dipendenze funzionali Dip_p , se ne calcola la sua **Chiusura Transitiva** $(Dip_p)^+$
Si estrae $Dip_p^0(A_0)$, l'insieme degli archi
 η of $A_0 \rightarrow \sigma$ of A_0 ($\eta \in Ere(A_0)$, $\sigma \in Sin(A_0)$).
- Per ogni simbolo non-terminale $A \in V$, si calcola
 $DD^0(A) = \cup_p \{Dip_p^0(A_0)\}$ per tutte le regole di
produzione p con $A_0 = A$ ($DD^0(A)$ è un **Insieme di Grafi**).

Passo Iterativo $i \geq 1$

- Per ogni regola di produzione $p : A_0 \rightarrow A_1 A_2 \dots A_n$ e per ogni combinazione di grafi $c = \langle D(A_1), \dots, D(A_n) \rangle$ con $D(A_j) \in DD^{i-1}(A_j)$
Si calcola $Dip'_{p,c} = Dip_p \cup_j D(A_j)$
e la sua chiusura transitiva $(Dip'_{p,c})^+$
da cui si estrae $Dip^i_{p,c}(A_0)$ l'insieme degli archi η of $A_0 \rightarrow \sigma$ of A_0 ($\eta \in Ere(A_0)$, $\sigma \in Sin(A_0)$).
- Per ogni non-terminale A , si calcola $DD^i(A) = DD^{i-1}(A) \cup_{p,c} \{Dip^i_{p,c}(A)\}$.

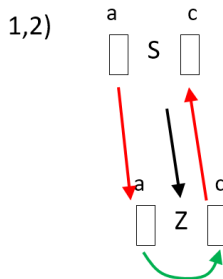
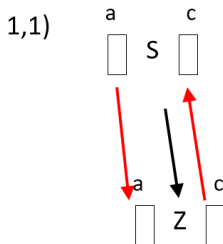
- Se succede che un $(Dip'_{p,c})^+$ contiene cicli,
 $\Rightarrow$ La grammatica non è aciclica.
- Se per almeno un non-terminale A succede che $DD^i(A) \neq DD^{i-1}(A)$, si ripete il passo iterativo altrimenti STOP (la grammatica è aciclica)

Riprendiamo la grammatica che non è assolutamente aciclica

Passo 0

- $DD^0(S) = \{Dip_1^0(S)\} = \{\emptyset\}$
- $DD^0(Z) = \{Dip_2^0(Z), Dip_3^0(Z)\} = \{\emptyset, \{a \rightarrow c\}\}$
- $DD^0(A) = \{Dip_4^0(A)\} = \{\emptyset\}$
- $DD^0(B) = \{Dip_5^0(B), Dip_6^0(B)\} = \{\{b \rightarrow c\}, \{a \rightarrow d\}\}$

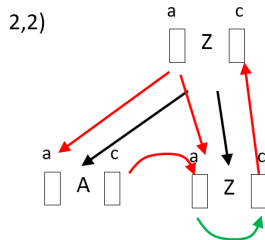
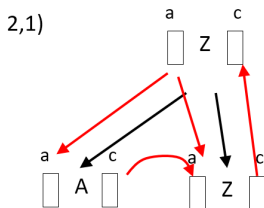
Passo 1



$$Dip_{1,1}^1(S) = \emptyset$$

$$Dip_{1,2}^1(S) = \{a \rightarrow c\}$$

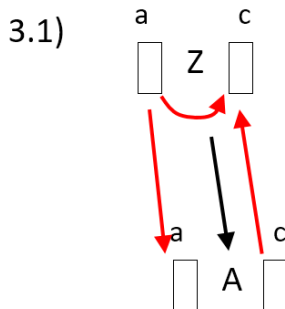
Passo 1



$$Dip_{2,1}^1(Z) = \emptyset$$

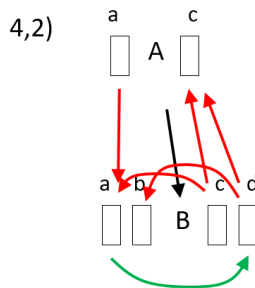
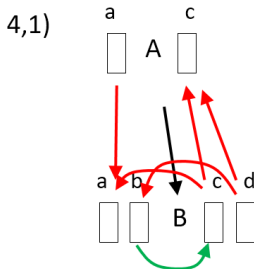
$$Dip_{2,2}^1(Z) = \{a \rightarrow c\}$$

Passo 1



$$Dip_{3,1}^1(Z) = \{a \rightarrow c\}$$

Passo 1



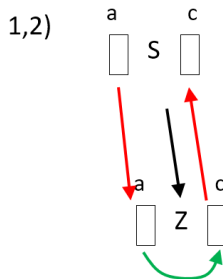
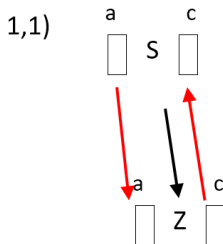
$$Dip_{4,1}^1(A) = \emptyset$$

$$Dip_{4,2}^1(A) = \{a \rightarrow c\}$$

Passo 1

- $DD^1(S) = DD^0(S) \cup \{Dip_{1,1}^1(S), Dip_{1,2}^1(S)\} =$
 $= \{\emptyset\} \cup \{\emptyset, \{a \rightarrow c\}\} = \{\emptyset, \{a \rightarrow c\}\}$ **cambiato**
- $DD^1(Z) =$
 $DD^0(Z) \cup \{Dip_{2,1}^1(Z), Dip_{2,2}^1(Z), Dip_{3,1}^1(Z)\} =$
 $= \{\emptyset, \{a \rightarrow c\}\} \cup \{\emptyset, \{a \rightarrow c\}, \{a \rightarrow c\}\} =$
 $= \{\emptyset, \{a \rightarrow c\}\}$
- $DD^1(A) = DD^0(A) \cup \{Dip_{4,1}^1(A), Dip_{4,2}^1(A)\} =$
 $= \{\emptyset\} \cup \{\emptyset, \{a \rightarrow c\}\} = \{\emptyset, \{a \rightarrow c\}\}$ **cambiato**
- $DD^1(B) = DD^0(B) \cup \{Dip_{5,1}^1(B), Dip_{6,1}^1(B)\} =$
 $= \{\{b \rightarrow c\}, \{a \rightarrow d\}\} \cup \{\{b \rightarrow c\}, \{a \rightarrow d\}\} =$
 $= \{\{b \rightarrow c\}, \{a \rightarrow d\}\}$

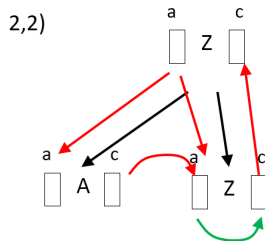
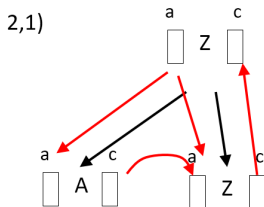
Passo 2



$$Dip_{1,1}^2(S) = \emptyset$$

$$Dip_{1,2}^2(S) = \{a \rightarrow c\}$$

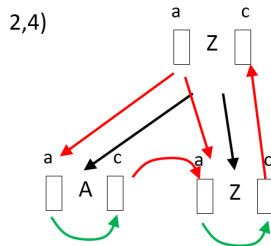
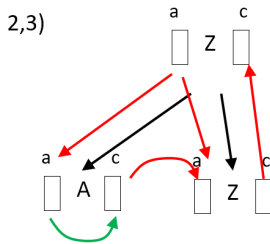
Passo 2



$$Dip_{2,1}^2(Z) = \emptyset$$

$$Dip_{2,2}^2(Z) = \{a \rightarrow c\}$$

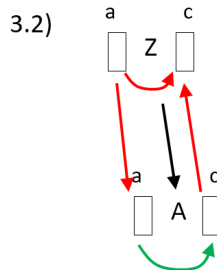
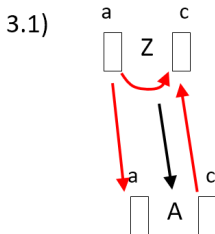
Passo 2



$$Dip_{2,3}^2(Z) = \emptyset$$

$$Dip_{2,4}^2(Z) = \{a \rightarrow c\}$$

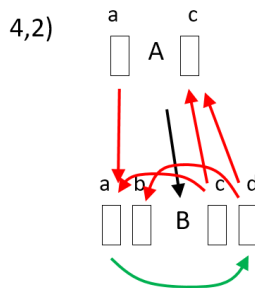
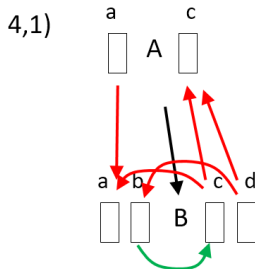
Passo 2



$$Dip_{3,1}^2(Z) = \{a \rightarrow c\}$$

$$Dip_{3,2}^2(Z) = \{a \rightarrow c\}$$

Passo 2



$$Dip_{4,1}^2(A) = \emptyset$$

$$Dip_{4,2}^2(A) = \{a \rightarrow c\}$$

Passo 2

- $DD^2(S) = DD^1(S) \cup \{Dip_{1,1}^2(S), Dip_{1,2}^2(S)\} =$
 $= \{\emptyset, \{a \rightarrow c\}\} \cup \{\emptyset, \{a \rightarrow c\}\} = \{\emptyset, \{a \rightarrow c\}\}$
- $DD^2(Z) = DD^1(Z) \cup \{Dip_{2,1}^2(Z), Dip_{2,2}^2(Z), Dip_{2,3}^2(Z),$
 $Dip_{2,4}^2(Z), Dip_{3,1}^2(Z), Dip_{3,2}^2(Z)\} =$
 $= \{\emptyset, \{a \rightarrow c\}\} \cup \{\emptyset, \{a \rightarrow c\}, \emptyset, \{a \rightarrow c\},$
 $\{a \rightarrow c\}, \{a \rightarrow c\}\} = \{\emptyset, \{a \rightarrow c\}\}$
- $DD^2(A) = DD^1(A) \cup \{Dip_{4,1}^2(A), Dip_{4,2}^2(A)\} =$
 $= \{\emptyset, \{a \rightarrow c\}\} \cup \{\emptyset, \{a \rightarrow c\}\} = \{\emptyset, \{a \rightarrow c\}\}$
- $DD^2(B) = DD^1(B) \cup \{Dip_{5,1}^2(B), Dip_{6,1}^2(B)\} =$
 $= \{\{b \rightarrow c\}, \{a \rightarrow d\}\} \cup \{\{b \rightarrow c\}, \{a \rightarrow d\}\} =$
 $= \{\{b \rightarrow c\}, \{a \rightarrow d\}\}$

Passo 2

- Nessun ciclo
- Nessun cambiamento, quindi la grammatica è aciclica in modo esatto (ma non era assolutamente aciclica)
- Prezzo da pagare: l'algoritmo esatto è esponenziale